

MIT 6.S191: Introduction to Deep Learning

Lecture 2 — Quick Reference Notes

Deep Sequence Modeling

Tayaba Abbasi

September 12, 2026

These notes were compiled while studying MIT 6.S191, *Deep Sequence Modeling*, and from personal lecture notes. They are intended as a concise revision and interview-preparation reference rather than a replacement for the official lecture.

Contents

1	Why Sequence Modeling?	3
1.1	Common Sequence Modeling Tasks	3
2	From Feed-Forward Networks to Recurrence	3
2.1	The Hidden State	4
3	Recurrent Neural Networks (RNNs)	4
3.1	Unrolling an RNN Through Time	4
4	RNN Intuition: Next-Word Prediction	5
5	Representing Language Numerically	5
5.1	Tokenization and Indices	5
5.2	One-Hot Representations	5
5.3	Embeddings	5
5.4	Design Criteria for Sequence Models	6
6	Training RNNs: Backpropagation Through Time	6
7	Vanishing and Exploding Gradients	6
7.1	Exploding Gradients	6
7.2	Vanishing Gradients	7
8	LSTMs and Better Memory Control	7

9 Fundamental Limitations of Recurrence	7
9.1 Encoding Bottleneck	7
9.2 Limited Parallelization	7
10 Attention: Learning What Matters	8
11 Positional Information	8
12 Self-Attention	8
12.1 Query-Key Similarity	8
12.2 Attention Weights	9
12.3 Weighted Values	9
13 Attention Heads and Transformers	9
14 RNNs vs. Attention: Compact Comparison	10
15 Lecture 2: Compact Mental Model	10
16 Interview Review Questions	10

1 Why Sequence Modeling?

Many real-world data sources are inherently ordered: text, audio, medical signals, financial time series, biological sequences, and motion trajectories. In such data, the meaning of the current observation often depends on what came before it.

A useful intuition is the lecture's moving-ball example. From a single image of a ball, its next position is ambiguous. Given several previous positions, however, the trajectory provides context that makes the next position easier to predict.

Key Idea

Sequence modeling is about learning from **ordered context**. Treating each time step independently discards information about relationships across time.

1.1 Common Sequence Modeling Tasks

Sequence problems can be organized according to the relationship between inputs and outputs:

- **One-to-one:** one input produces one output, e.g. binary classification.
- **Many-to-one:** a sequence produces one output, e.g. sentiment classification.
- **One-to-many:** one input produces a sequence, e.g. image captioning.
- **Many-to-many:** one sequence maps to another sequence, e.g. machine translation.

Interview Check

Q: Why is sequence modeling different from ordinary feed-forward prediction?

In sequential data, observations are not independent. The interpretation or prediction at time t may depend on observations from earlier time steps, so the model needs a mechanism for representing temporal or positional dependencies.

2 From Feed-Forward Networks to Recurrence

A feed-forward network can process an input x_t at a particular time step and produce

$$\hat{y}_t = f(x_t).$$

If the same network is applied independently to $x_0, x_1, x_2, \dots$, each prediction is still isolated from the others. There is no memory of previous inputs.

For sequential modeling, we instead want

$$\hat{y}_t = f(x_t, h_{t-1}),$$

where h_{t-1} represents information carried forward from previous computations.

2.1 The Hidden State

The hidden state h_t acts as the network's internal memory. It is updated as the sequence is processed:

$$h_t = f_W(x_t, h_{t-1}).$$

The same recurrence function and the same learned parameters W are reused at every time step.

Intuition

An RNN is not a new neural network for every word or time step. It is **one network reused through time**, with its hidden state carrying information from earlier steps into later computations.

3 Recurrent Neural Networks (RNNs)

A simple recurrent neural network updates its state using the current input and previous hidden state. A common formulation is

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h),$$

followed by an output transformation

$$\hat{y}_t = W_{hy}h_t + b_y.$$

Depending on the task, an additional activation such as softmax may be applied to the output.

The learnable parameter sets include:

- W_{xh} : input-to-hidden transformation,
- W_{hh} : hidden-to-hidden recurrent transformation,
- W_{hy} : hidden-to-output transformation.

3.1 Unrolling an RNN Through Time

The recurrent loop can be visualized as an unrolled computational graph:

$$(x_0, h_{-1}) \rightarrow h_0 \rightarrow (x_1, h_0) \rightarrow h_1 \rightarrow \dots \rightarrow h_T.$$

Although the diagram appears to contain many copies of the RNN cell, the parameters are **shared** across all time steps.

Interview Check

Q: Why are RNN weights shared across time?

The same transformation is applied at every position in the sequence. Parameter sharing allows the model to learn a reusable rule for processing sequential inputs and avoids learning a separate model for every possible time step.

4 RNN Intuition: Next-Word Prediction

Consider the partial sentence

I love recurrent neural ...

The RNN processes one token at a time. At every step, it updates its hidden state using the current word and the state from the previous step. After processing the available context, the latest output can be used to predict the next word.

Conceptually:

```
hidden_state = initial_state
for word in sentence:
    prediction, hidden_state = rnn(word, hidden_state)
next_word_prediction = prediction
```

Key Idea

The prediction after the final observed word depends indirectly on all earlier words through the evolving hidden state.

5 Representing Language Numerically

Neural networks operate on numerical inputs, so text must be converted into numerical representations before it can be processed.

5.1 Tokenization and Indices

A vocabulary can assign each token a unique index. This provides a numerical identity but does not by itself represent semantic similarity.

5.2 One-Hot Representations

A one-hot vector represents a word using a vector with a single 1 and zeros elsewhere. This distinguishes words, but two semantically related words are no closer than two unrelated words.

5.3 Embeddings

Embeddings map tokens into dense continuous vectors. Ideally, the learned space captures meaningful relationships, so semantically related words occupy nearby regions.

Key Idea

A useful representation should do more than identify a token. It should capture some of the structure and relationships present in language.

5.4 Design Criteria for Sequence Models

The lecture motivates four important requirements:

1. map raw sequence elements into numerical representations;
2. handle variable-length sequences;
3. capture dependencies between elements;
4. preserve the importance of order.

Order matters because changing token order can change meaning. For example, “good, not bad” and “bad, not good” contain similar words but communicate different sentiments.

6 Training RNNs: Backpropagation Through Time

As in a feed-forward network, training begins with a forward pass and a loss. In an RNN, however, computation extends across multiple time steps.

If a loss is computed at each step,

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t,$$

then the gradients must flow backward through the unrolled sequence. This procedure is called **Backpropagation Through Time (BPTT)**.

Because the same recurrent parameters are reused at every step, gradients with respect to earlier states involve repeated products involving recurrent weights and activation derivatives.

Interview Check

Q: How is BPTT different from ordinary backpropagation?

The underlying chain rule is the same. BPTT applies backpropagation to an RNN unrolled across time, so gradients propagate not only through layers but also backward across sequential time steps.

7 Vanishing and Exploding Gradients

Repeated multiplication during BPTT can make gradients numerically unstable.

7.1 Exploding Gradients

If repeated factors have sufficiently large magnitude, gradients can grow rapidly:

$$\left\| \frac{\partial \mathcal{L}}{\partial h_t} \right\| \gg 1.$$

This can produce unstable parameter updates. A common mitigation is **gradient clipping**, which limits the gradient magnitude before the optimizer update.

7.2 Vanishing Gradients

If repeated factors are small, gradients can shrink toward zero:

$$\left\| \frac{\partial \mathcal{L}}{\partial h_t} \right\| \approx 0.$$

Then early time steps receive very little learning signal, making long-range dependencies difficult to learn.

Interview Check

Q: Why do vanilla RNNs struggle with long-term dependencies?

During BPTT, gradients are repeatedly multiplied as they travel through many time steps. They may vanish, preventing useful learning from distant context, or explode and become unstable.

8 LSTMs and Better Memory Control

One response to the limitations of vanilla RNNs is to make the recurrent state update more structured. Long Short-Term Memory (LSTM) networks introduce mechanisms that better control which information is retained, updated, or discarded.

The lecture uses LSTMs primarily to motivate the broader goal: improving information flow so that important dependencies can persist over longer sequences.

Key Idea

LSTMs improve the management of recurrent memory, but they do not remove the fundamental sequential nature of recurrence: time steps are still processed in order.

9 Fundamental Limitations of Recurrence

RNN-style sequence modeling introduces two major bottlenecks.

9.1 Encoding Bottleneck

The hidden state must compress relevant information from the past into a fixed internal representation. As sequences become long and complex, preserving every useful dependency becomes difficult.

9.2 Limited Parallelization

To compute h_t , an RNN needs h_{t-1} . Therefore, later time steps cannot be fully processed before earlier states are available. This sequential dependency makes RNN computation difficult to parallelize efficiently.

These limitations motivate a different question: can a model capture relationships across a sequence **without recurrence**?

10 Attention: Learning What Matters

Attention provides a mechanism for a neural network to identify which parts of an input are most relevant to representing or predicting another part.

The lecture's search analogy is useful:

- **Query:** what am I looking for?
- **Key:** what information describes each candidate item?
- **Value:** what content should I retrieve if that item is relevant?

The model compares a query against keys, converts the similarities into attention weights, and uses those weights to combine the corresponding values.

Key Idea

Attention replaces the idea of carrying all useful history through one recurrent state with the ability to directly weight relevant parts of the sequence.

11 Positional Information

If recurrence is removed and sequence elements are processed together, the architecture still needs information about order. Positional encodings or embeddings inject information about where each element occurs in the sequence.

If e_t is a token embedding and p_t is its positional representation, the model can operate on

$$z_t = e_t + p_t.$$

The exact positional-encoding construction is not the focus of this lecture; the important point is that attention-based models need an explicit way to retain positional relationships.

12 Self-Attention

In self-attention, the model learns relationships **within the same sequence**. Starting from an input representation matrix X , learned linear transformations produce queries, keys, and values:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Although all three originate from the same input representation, they use different learned transformations and therefore play different roles.

12.1 Query-Key Similarity

Similarity between queries and keys is computed using dot products. In scaled dot-product attention,

$$S = \frac{QK^T}{\sqrt{d_k}},$$

where d_k is the key dimension.

The scaling helps keep dot-product magnitudes controlled as dimensionality increases.

12.2 Attention Weights

Softmax converts the similarity scores into normalized attention weights:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right).$$

Large entries indicate stronger relevance between positions.

12.3 Weighted Values

The attention output is obtained by applying the attention weights to the values:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

Intuition

Query asks what information is needed. **Keys** determine where relevant information may be found. **Values** contain the information that is actually aggregated into the output.

Interview Check

Q: What is self-attention?

Self-attention allows each position in a sequence to compute a weighted combination of information from other positions in that same sequence. The weights are learned dynamically from query-key similarity.

13 Attention Heads and Transformers

The sequence of operations

$$X \rightarrow (Q, K, V) \rightarrow QK^T \rightarrow \text{scale} \rightarrow \text{softmax} \rightarrow AV$$

forms an **attention head**.

Modern transformer architectures use attention as a core building block. Multiple attention heads can be used in parallel so that different heads can learn different patterns of relationships in the same sequence.

Compared with recurrent processing, attention allows much more parallel computation because the sequence can be considered globally rather than strictly one time step after another.

Interview Check**Q: Why did attention become so important for modern sequence models?**

It provides direct access to relationships between positions, reduces dependence on a single recurrent memory state, and is much more parallelizable than step-by-step recurrence. These properties make it well suited to large-scale transformer models.

14 RNNs vs. Attention: Compact Comparison

Property	RNN	Attention-based model
Sequence processing	Step by step	Sequence considered globally
Memory	Hidden state h_t	Direct weighted interactions
Parameter use	Shared across time	Learned Q/K/V projections
Long dependencies	Can be difficult	Direct position-to-position access
Parallelization	Limited	Much stronger
Order information	Built into recurrence	Added explicitly via position

15 Lecture 2: Compact Mental Model

The progression of the lecture can be summarized as:

Sequence Data $\rightarrow$ Recurrence $\rightarrow$ RNN Hidden State $\rightarrow$ BPTT
 $\rightarrow$ Gradient Problems $\rightarrow$ Attention $\rightarrow$ Transformers

A practical mental model is:

1. Sequence data contains dependencies across ordered elements.
2. Feed-forward processing alone treats time steps independently.
3. RNNs introduce a hidden state to carry information through time.
4. RNNs reuse the same parameters at every time step.
5. BPTT trains the recurrent network across the unrolled sequence.
6. Repeated gradient multiplication creates vanishing/exploding-gradient problems.
7. Long sequences also expose a hidden-state bottleneck and poor parallelization.
8. Attention directly learns which positions are relevant to one another.
9. Self-attention uses learned queries, keys, and values.
10. Transformers build on attention and enable highly parallel sequence modeling.

16 Interview Review Questions

1. What makes sequence data different from ordinary tabular inputs?

2. What are one-to-one, many-to-one, one-to-many, and many-to-many sequence tasks?
3. Why does independently applying a feed-forward network at each time step lose useful information?
4. What does the hidden state of an RNN represent?
5. Write the basic RNN hidden-state update equation.
6. Why are RNN parameters shared across time steps?
7. How can an RNN be used for next-word prediction?
8. Why do text inputs require tokenization and numerical representations?
9. What advantage do embeddings have over one-hot vectors?
10. What is Backpropagation Through Time?
11. What causes exploding gradients, and how can gradient clipping help?
12. What causes vanishing gradients?
13. Why do vanishing gradients make long-term dependencies difficult to learn?
14. What problem are LSTMs intended to address?
15. What is the hidden-state or encoding bottleneck in RNNs?
16. Why are RNNs difficult to parallelize?
17. What is the intuition behind attention?
18. What roles do the query, key, and value play in attention?
19. Why is positional information needed when recurrence is removed?
20. What is self-attention?
21. Write the scaled dot-product attention equation.
22. Why is softmax applied to attention scores?
23. What does an attention matrix represent?
24. What is an attention head?
25. Why are attention-based models more parallelizable than RNNs?
26. How does attention help with long-range dependencies?

Personal quick-reference notes based on MIT 6.S191 Lecture 2 and personal lecture notes.